

Blog

WordPress Web Application & Host Security Assessment

CONFIDENTIAL

Engagement Type	Black-box assessment
Target	blog.thm (10.48.190.74)
Application	"Billy Joel's IT Blog" — WordPress 5.0
Exposed Services	HTTP (80), SMB (139/445)
Host OS	Linux / Ubuntu
Tester	Saqr Elfirgany
Report Date	25 June 2026
Environment	TryHackMe — authorised lab

This document contains confidential information about the security posture of the assessed target. The engagement was performed in an isolated, authorised training environment (TryHackMe). The techniques documented here must only be used against systems you are explicitly authorised to test.

Table of Contents

1. Executive Summary	3
2. Scope & Methodology	3
3. Findings Summary	4
4. Attack Narrative (Kill Chain)	4
5. Detailed Findings	5
6. Remediation Summary	8
7. Appendix A — Services & Ruled-Out Paths	9

1. Executive Summary

A black-box penetration test was performed against the host `blog.thm`, which hosts a WordPress 5.0 site ("Billy Joel's IT Blog"). Starting unauthenticated, the tester enumerated valid users, recovered a weak password, exploited a known WordPress core vulnerability to gain code execution, and escalated to **root** via an insecure custom SUID binary — a full compromise of the host.

The decisive weakness was an **outdated WordPress core (5.0)** affected by the well-documented authenticated crop-image RCE chain (CVE-2019-8942 and CVE-2019-8943). Reaching it was made easy by **username enumeration** (both via the login form and author data leaked in the RSS feed) and a **weak account password with no brute-force protection**. Once on the host, privilege escalation was immediate due to a **custom SUID binary that trusts an attacker-controllable environment variable**. Additional lower-risk issues — stored/DOM XSS, directory listing, and permissive SMB/XML-RPC exposure — were also identified.

Overall Risk: Critical. A remote attacker can progress from no access to full root control. The outdated WordPress core (F-01) and the insecure SUID binary (F-02) are the two highest-priority items; together they form the complete path to root.

2. Scope & Methodology

Scope

A single host (`blog.thm`) was in scope, assessed from an unauthenticated, external (black-box) perspective.

Methodology

The assessment followed a structured, surface-first methodology, separating enumeration from exploitation:

- **Reconnaissance** — port/service scanning; web and SMB enumeration.

- **Enumeration** — WordPress fingerprinting, user enumeration, content discovery.
- **Attack-surface mapping** — each lead recorded and classified as working, ruled-out, or informational before exploitation.
- **Exploitation** — credential recovery and the WordPress core RCE chain.
- **Post-exploitation** — local enumeration and privilege escalation to root.

3. Findings Summary

ID	FINDING	SEVERITY	AFFECTED COMPONENT
F-01	Outdated WordPress core — authenticated RCE (CVE-2019-8942 / 8943)	CRITICAL	WordPress 5.0
F-02	Privilege escalation via insecure SUID binary	HIGH	/usr/sbin/checker
F-03	Weak credentials & no brute-force protection	HIGH	wp-login.php
F-04	Stored / DOM-based Cross-Site Scripting	MEDIUM	WordPress app
F-05	Username enumeration (login form + RSS author disclosure)	MEDIUM	wp-login.php, /feed/
F-06	Directory listing enabled on uploads	LOW	/wp-content/uploads/
F-07	Permissive SMB guest access & exposed XML-RPC	LOW	SMB, /xmlrpc.php

4. Attack Narrative (Kill Chain)

1. Scanning revealed a WordPress site on port 80 and Samba on 139/445. The blog was fingerprinted as **WordPress 5.0** (confirmed via `/feed/`).
2. The RSS feed and site content leaked author identities (Billy Joel, Karen Wheeler), and the login form confirmed a valid username through its error wording — establishing `bjoel` and `kwheel` as valid accounts (**F-05**).
3. With no brute-force protection, a weak password for `kwheel` was recovered: `cutiepie1` (**F-03**).
4. Authenticated as `kwheel`, the **WordPress 5.0 crop-image RCE chain (F-01)** — CVE-2019-8942 (post-meta abuse) plus CVE-2019-8943 (crop-image path traversal) — was used to write a PHP payload to the web root, yielding a reverse shell as `www-data`.
5. Local enumeration found a non-standard **SUID binary**, `/usr/sbin/checker` (**F-02**), which grants a root shell when an attacker-set `admin` environment variable is present.
6. Setting `admin=1` and invoking the binary returned a **root** shell, completing host compromise.

5. Detailed Findings

Outdated WordPress Core — Authenticated RCE (CVE-2019-8942 / 8943)

F-01 ·
CRITICAL

Severity: **CRITICAL** | **Component:** WordPress 5.0 core | **Type:** Known Vulnerable Component / Remote Code Execution (CWE-1035 / CWE-22 / CWE-434)

Description

The site runs WordPress 5.0, which is affected by a well-documented authenticated remote-code-execution chain. CVE-2019-8942 allows a user with at least Author privileges to manipulate the `_wp_attached_file` post-meta value, and CVE-2019-8943 exploits a path traversal in the image-crop feature to write a PHP file into the web root. Chained, they allow arbitrary code execution as the web-server account.

Evidence

```
Authenticated as kwheel:cutiepie1
Exploit: WordPress crop-image RCE (CVE-2019-8942 + CVE-2019-8943)
→ PHP payload written to web root → reverse shell as www-data
```

Impact

Full remote code execution on the host as `www-data`, providing the foothold for privilege escalation.

Remediation

- Upgrade WordPress core to a current, supported release and keep it patched.
- Restrict author/editor accounts and apply least privilege within WordPress roles.
- Deploy file-integrity monitoring on the web root to detect unexpected PHP files.

Privilege Escalation via Insecure SUID Binary

F-02 · HIGH

Severity: HIGH | **Component:** /usr/sbin/checker (SUID) | **Type:** Insecure SUID / External Control of Setting (CWE-250 / CWE-15)

Description

A non-standard, root-owned SUID binary, `/usr/sbin/checker`, made a privileged decision based on an environment variable (`admin`) that any local user can set. By defining the variable before execution, an unprivileged user obtains a root shell.

Evidence

```
find / -perm -4000 -type f 2>/dev/null → /usr/sbin/checker
export admin=1
/usr/sbin/checker
id → uid=0(root)
```

Impact

Immediate, reliable escalation from `www-data` to full root privileges.

Remediation

- Remove the SUID bit from custom binaries unless strictly required.
- Never base privilege decisions on user-controllable environment variables; sanitise the environment in setuid programs.
- Audit all SUID binaries against the expected OS baseline and remove non-standard ones.

Weak Credentials & No Brute-Force Protection

F-03 · HIGH

Severity: HIGH | **Component:** wp-login.php | **Type:** Weak Password / Excessive Auth Attempts (CWE-521 / CWE-307)

Description

The WordPress login enforced no rate limiting or lockout, and the `kwheel` account used a weak, guessable password. This combination allowed the credentials to be recovered, providing the authenticated access that the RCE chain (F-01) requires.

Evidence

```
Recovered credentials: kwheel : cutiepie1
```

Impact

Authenticated access to WordPress, which is the precondition for the critical RCE.

Remediation

- Enforce a strong password policy and screen against breached-password lists.
- Add login rate limiting / lockout and consider multi-factor authentication.
- Limit XML-RPC, which can be abused to amplify credential attacks (see F-07).

Stored / DOM-Based Cross-Site Scripting

F-04 · MEDIUM

Severity: MEDIUM | **Component:** WordPress application | **Type:** Cross-Site Scripting (CWE-79)

Description

Cross-site scripting (stored and DOM-based) was identified within the application during testing. Although it was not used in the path to root, it represents a genuine exposure: an attacker could execute script in the context of other users' sessions, potentially leading to session theft or actions performed on their behalf.

Evidence

XSS sinks were observed during manual review of the application. The issue was identified but not weaponised, as a more direct path to compromise (F-01) was available.

Impact

Potential session hijacking, credential theft, or unauthorised actions in the browser context of victims, including administrators.

Remediation

- Apply context-aware output encoding and input validation across all user-controllable fields.
- Deploy a Content-Security-Policy and set appropriate cookie flags (HttpOnly, Secure, SameSite).
- Keep WordPress core, themes, and plugins updated, as many XSS issues are fixed upstream.

Username Enumeration (Login Form + RSS Author Disclosure)

F-05 · MEDIUM

Severity: MEDIUM | **Component:** wp-login.php, /feed/ | **Type:** Observable Response Discrepancy / Information Exposure (CWE-204 / CWE-200)

Description

Valid usernames could be confirmed through two channels: the login form returned a message specific to a valid username with an incorrect password, and the RSS feed (/feed/) disclosed author names that mapped to account logins.

Evidence

```
wp-login.php → "The password you entered for the username kwheel is incorrect"
/feed/       → leaked authors: Billy Joel, Karen Wheeler → bjoel, kwheel
```

Impact

Confirmed valid usernames make password attacks far more efficient and support targeted phishing.

Remediation

- Return generic authentication errors that do not reveal username validity.
- Limit author-data exposure in feeds and REST endpoints where feasible.

Directory Listing Enabled on Uploads

F-06 · LOW

Severity: LOW | **Component:** /wp-content/uploads/ | **Type:** Information Disclosure (CWE-548)

Description

The uploads directory permitted directory listing, exposing the names and structure of stored files to unauthenticated visitors.

Evidence

```
http://blog.thm/wp-content/uploads/ → directory index displayed
```

Impact

Exposure of potentially sensitive uploaded files and additional reconnaissance value for an attacker.

Remediation

- Disable automatic directory indexing on the web server (e.g. `Options -Indexes`).
- Place an index file in upload directories and restrict access to sensitive content.

Permissive SMB Guest Access & Exposed XML-RPC

F-07 · LOW

Severity: LOW | **Component:** SMB (139/445), /xmlrpc.php | **Type:** Security Misconfiguration (CWE-16)

Description

The SMB service (Samba 4.7.6) permitted guest access and did not require message signing, and the WordPress `xmlrpc.php` interface was exposed. While neither yielded direct compromise here, both expand the attack surface — XML-RPC in particular can be abused to amplify credential-guessing and for pingback-based attacks.

Evidence

```
SMB: account_used: guest; message signing disabled (not required)
/xmlrpc.php: XML-RPC server present (accepts POST)
```

Impact

Increased attack surface and potential for credential-attack amplification and information gathering.

Remediation

- Disable guest access and require SMB message signing; restrict SMB to trusted networks.
- Disable or restrict `xmlrpc.php` if not required.

6. Remediation Summary

ID	PRIORITY ACTION	PRIORITY
F-01	Upgrade WordPress core to a supported version; patch regularly.	IMMEDIATE

ID	PRIORITY ACTION	PRIORITY
F-02	Remove SUID from custom binaries; never trust user-controlled env vars in setuid code.	HIGH
F-03	Enforce strong passwords and add login rate limiting / MFA.	HIGH
F-04	Fix XSS via output encoding and CSP; keep components updated.	MEDIUM
F-05	Return generic auth errors; limit author disclosure in feeds.	MEDIUM
F-06	Disable directory indexing on the web server.	LOW
F-07	Harden SMB (no guest, require signing); restrict XML-RPC.	LOW

7. Appendix A — Services & Ruled-Out Paths

Discovered Services

PORT	SERVICE	VERSION / NOTES
80/tcp	HTTP	WordPress 5.0 — "Billy Joel's IT Blog"
139/445	SMB	Samba 4.7.6-Ubuntu — guest access, signing not required

Ruled-Out Paths & Informational Leads

The following were investigated and either ruled out or noted as informational. Documenting them is part of mapping the full attack surface.

LEAD	OUTCOME
SMB shares (guest)	Ruled out — nothing useful
robots.txt / /wp-includes/ / wp-admin password reset	Ruled out
Internal IP (192.168.196.139)	Ruled out — not the route
RSS feed /feed/	Informational — leaked author names
/xmlrpc.php, /wp-json/	Informational — exposed interfaces
Uploads directory listing	Informational — see F-06

Tester's note. Leads were classified during testing as *working*, *ruled-out*, or *informational*, so effort stayed focused on the most promising paths. Identifying issues outside the primary kill chain (e.g. the XSS in F-04) reflects assessing the whole attack surface rather than only the route to root.